LAPORAN PRAKTIKUM PEKAN 5

STRUKTUR DATA

“Single Linked List”

disusun Oleh:

NABIL FIKRI

2511533011

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

SHERLY SUKMADIRA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum mata kuliah **Struktur Data** ini tepat pada waktunya.

Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum mengenai implementasi kelas dan objek melalui studi kasus pembuatan **Single Linked List** menggunakan bahasa pemrograman Java. Penulis menyadari bahwa keberhasilan penyusunan laporan ini tidak terlepas dari bantuan berbagai pihak. Oleh karena itu, penulis ingin menyampaikan terima kasih kepada:

1. Dr. Wahyudi S.T,M.T, selaku dosen mata kuliah yang telah memberikan bimbingan teori di kelas.
2. Kakak asisten praktikum yang telah memberikan arahan dan bantuan teknis selama proses praktikum berlangsung di laboratorium.
3. Rekan-rekan mahasiswa angkatan 2025 Informatika Universitas Andalas yang telah saling mendukung dalam proses belajar.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan, baik dari segi materi maupun sistematika penulisannya. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan perkembangan ilmu pengetahuan, khususnya di bidang Informatika.

Padang, 06 Mei 2026

Nabil Fikri

DAFTAR ISI

KATA PENGANTAR i

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan Praktikum 2

BAB II PEMBAHASAN 3

2.1 Single Linked List 3

2.1.1 Pengertian 3

2.1.2 Operasi 3

2.1.3 Kelebihan dan Kekurangan Single Linked List 4

2.2 Implementasi pada java 4

2.2.1 Class NodeSLL_2511533011 4

2.2.2 PencarianSLL_2511533011 6

2.2.3 TambahSLL_2511533011 8

2.2.4 Class HapusSLL_2511533011 11

BAB III PENUTUP 15

3.1 Kesimpulan 15

3.2 Saran 15

DAFTAR PUSTAKA 16

LAPORAN TUGAS 17

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam disiplin ilmu Informatika, pengelolaan data secara dinamis merupakan salah satu aspek fundamental dalam membangun aplikasi yang efisien. Struktur data **Array** sering kali menjadi pilihan utama karena kemudahannya, namun Array memiliki batasan pada alokasi memori statis, di mana ukurannya harus ditentukan di awal. Hal ini menyebabkan ketidakefisienan ketika jumlah data yang diolah bersifat fluktuatif, yang berujung pada pemborosan memori atau risiko *overflow.*

Sebagai solusi atas keterbatasan tersebut, diperkenalkan struktur data **Single Linked List (SLL)**. SLL adalah sekumpulan simpul (*node*) yang terhubung secara satu arah melalui referensi atau penunjuk. Dalam bahasa pemrograman **Java**, konsep ini diimplementasikan menggunakan referensi objek. Setiap simpul (yang direpresentasikan oleh class NodeSLL_2511533011) menyimpan data dan alamat dari simpul berikutnya.

Implementasi SLL pada Java memiliki keunikan tersendiri karena adanya fitur *Garbage Collection*, yang secara otomatis mengelola pembebasan memori ketika sebuah simpul tidak lagi memiliki referensi. Pemahaman mendalam mengenai manipulasi *pointer* melalui operasi penambahan (*insertion*), penghapusan (*deletion*), serta pencarian (*searching*) sangat krusial bagi mahasiswa Informatika sebagai fondasi untuk mempelajari struktur data yang lebih kompleks seperti *Stack, Queue,* dan *Trees*. Melalui praktikum pekan ke-5 ini, teori-teori tersebut diuji melalui implementasi kode nyata untuk memahami perilaku memori dinamis secara praktis.

1.2 Tujuan Praktikum

1. Mahasiswa mampu merancang class simpul menggunakan prinsip pemrograman berbasis objek di Java.
2. Mahasiswa mampu mengimplementasikan dan membedakan logika penambahan simpul di posisi depan (*front*), akhir (*end*), maupun posisi tertentu (*position*) di tengah list.
3. Mahasiswa mampu melakukan manipulasi referensi untuk menghapus simpul pada berbagai posisi dan memahami dampaknya terhadap alokasi memori.
4. Mahasiswa mampu menerapkan metode *Linear Search* untuk menemukan data spesifik di dalam daftar sekuensial.
5. Mahasiswa dapat menjelaskan perbedaan cara kerja akses data antara *Linked List* yang bersifat sekuensial dibandingkan dengan *Array* yang bersifat *random access*.

BAB II
PEMBAHASAN

2.1 Single Linked List

2.1.1 Pengertian

Linked List merupakan koleksi linear dari data, yang disebut sebagai *nodes,* dimana setiap *node* akan menunjuk pada *node* lain melalui sebuah *pointer*. *Linked List* dapat didefinisikan pula sebagai kumpulan *nodes* yang merepresentasikan sebuah *sequence.*

2.1.2 Operasi

a. Push

Push merupakan sebuah operasi *insert* dimana di dalam linked list terdapat 2 kemungkinan insert, yaitu insert melalui depan (pushDepan) ataupun belakang (pushBelakang). Operasi pushDepan berarti data yang paling baru dimasukkan akan berada di depan data lainnya, dan sebaliknya pushBelakang berarti data yang paling baru akan berada di belakang data lainnya.

b. Pop

Pop, kebalikan dari push, merupakan operasi *delete,* dimana di dalam *linked list* memiliki 2 kemungkinan *delete,* yaitu melalui depan (popDepan) dan melalui belakang (popBelakang). PopDepan berarti data yang akan dihapus adalah data paling depan, dan popBelakang berarti data yang akan dihapus adalah data paling belakang (akhir).

2.1.3 Kelebihan dan Kekurangan Single Linked List

Kelebihan Single Linked List :

1. Alokasi memori dinamis
2. Efisiensi operasi penambahan dan pengurangan
3. Tidak ada pemborosan memori (Memory Flexibility)

Kekuangan :

1. Penggunaan memori yang lebih besar (Memory Overhead)
2. Akses data bersifat Sekuensial
3. Penelusuran hanya satu arah
4. Rerentanan terhadap pointer

2.2 Implementasi pada java

2.2.1 Class NodeSLL_2511533011

```java
package Pekan5_2511533011;

public class NodeSLL_2511533011 {
    //node bagian data
    int data_3011;
    //Pointer ke node berikutnya
    NodeSLL_2511533011 next_3011;
    //Konstruktor menginisialisasi node dengan data
    public NodeSLL_2511533011(int data_3011)
    {
        this.data_3011 = data_3011;
        this.next_3011 = null;
    }
}
```

Gambar 2.2.1

Class ini merepresentasikan sebuah **Node** (simpul) dalam *Single Linked List*. Dalam konsep memori dinamis, *node* adalah sebuah objek yang membungkus informasi serta menjaga keterhubungan antar elemen.

1. Atribut Data (int data_3011)

Atribut ini berfungsi untuk menyimpan nilai atau informasi aktual yang ingin dikelola.

- Pada kode ini, tipe data yang digunakan adalah int, yang berarti setiap simpul dalam *list* akan menyimpan satu nilai bilangan bulat.
- Penamaan unik dengan akhiran _3011 (berdasarkan NIM) berfungsi sebagai identitas kepemilikan kode yang membedakannya dari variabel standar.

2. Atribut Pointer/Referensi (NodeSLL_2511533011 next_3011)

Ini adalah komponen paling krusial dalam sebuah *Linked List*.

- Atribut ini bertipe sama dengan nama *class*-nya sendiri, yang dikenal dengan istilah **Self-Referential Structure**.
- Fungsinya bukan menyimpan data, melainkan menyimpan **alamat memori** (referensi) dari *node* berikutnya. Inilah yang memungkinkan data tersebar di memori namun tetap saling terhubung secara logis dalam satu rantai.

3. Konstruktor (public NodeSLL_2511533011(int data_3011))

Konstruktor adalah metode khusus yang dijalankan saat objek *node* baru diciptakan (instansiasi).

- **Inisialisasi Data:** Mengambil nilai dari parameter dan memasukkannya ke dalam atribut data_3011.
- **Inisialisasi Referensi:** Secara *default*, saat sebuah *node* baru dibuat, atribut next_3011 diatur ke nilai null. Hal ini menandakan bahwa *node* tersebut berdiri sendiri dan belum menunjuk ke simpul manapun sebelum disambungkan oleh *class* pengelola *list*.

2.2.2 **PencarianSLL_2511533011**

```java
package Pekan5_2511533011;

public class PencarianSLL_2511533011 {
    static boolean searchKey(NodeSLL_2511533011 head_3031, int key_3031) {
        NodeSLL_2511533011 curr_3011 = head_3031;
        while (curr_3011 != null) {
            if (curr_3011.data_3011 == key_3031)
                return true;
            curr_3011 = curr_3011.next_3011; }
        return false; }
    public static void traversal(NodeSLL_2511533011 head_3011) {
        // mulai dari head
        NodeSLL_2511533011 curr_3011 = head_3011;
        // telusuri sampai pointer null
        while (curr_3011 != null) {
            System.out.print(" " + curr_3011.data_3011);
            curr_3011 = curr_3011.next_3011; }
```

```java
        System.out.println();}
    public static void main(String[] args) {
        NodeSLL_2511533011 head_3011 = new NodeSLL_2511533011(14);
        head_3011.next_3011 = new NodeSLL_2511533011(21);
        head_3011.next_3011.next_3011 = new NodeSLL_2511533011(13);
        head_3011.next_3011.next_3011.next_3011 = new NodeSLL_2511533011(30);
        head_3011.next_3011.next_3011.next_3011.next_3011 = new NodeSLL_2511533011(10);
        System.out.print("Penelusuran SLL : ");
        traversal(head_3011);
        // data yang akan dicari
        int key_3011 = 30;
        System.out.print("cari data " + key_3011 + " = ");
        if (searchKey(head_3011, key_3011))
            System.out.println("ketemu");
        else
            System.out.println("tidak ada");
```

Gambar 2.2.2

Class **PencarianSLL_2511533011** berfungsi sebagai kelas operasional sekaligus kelas utama (*driver class*) untuk mendemonstrasikan bagaimana data di dalam *Single Linked List* diakses, ditelusuri, dan dicari.

1. Method searchKey(NodeSLL_2511533011 head_3031, int key_3031)

Method ini mengimplementasikan algoritma **Linear Search** pada struktur *Linked List*.

- **Logika Kerja:** Method ini menggunakan variabel bantuan curr_3011 untuk menunjuk ke node saat ini, dimulai dari head. Program akan terus melakukan perulangan (while) selama curr_3011 belum mencapai null.
- **Kondisi Berhenti:**

- Jika curr_3011.data_3011 == key_3031, maka data ditemukan dan method mengembalikan nilai **true**.
- Jika perulangan selesai tanpa menemukan kecocokan, maka method mengembalikan nilai **false**.

- **Kompleksitas:** Operasi ini memiliki efisiensi waktu $O(n)$, karena dalam skenario terburuk, program harus memeriksa setiap node satu per satu.

2. Method traversal(NodeSLL_2511533011 head_3011)

Method ini berfungsi untuk mencetak seluruh isi data yang ada di dalam rantai *Linked List* ke konsol.

- **Mekanisme:** Program memulai dari node pertama (head) dan mencetak datanya. Setelah mencetak, posisi curr_3011 berpindah ke node berikutnya menggunakan perintah curr_3011 = curr_3011.next_3011.
- **Output:** Memberikan representasi visual urutan data kepada pengguna agar bisa memverifikasi struktur list yang telah dibuat.

3. Method main(String[] args)

Bagian ini adalah tempat pengujian semua logika yang telah dibangun.

- **Instansiasi Manual:** Di sini Anda membuat rantai list secara manual dengan menghubungkan referensi next_3011 secara bertingkat.
  - *Node 1 (14) -> Node 2 (21) -> Node 3 (13) -> Node 4 (30) -> Node 5 (10).*
- **Eksekusi:** Method ini memanggil traversal untuk menampilkan urutan angka tersebut, kemudian memanggil searchKey untuk mencari angka spesifik (dalam kode ini adalah angka **30**) untuk memastikan logika pencarian berfungsi dengan benar.

2.2.3 **TambahSLL_2511533011**

```java
public class TambahSLL_2511533011 {

    public static NodeSLL_2511533011 InsertAtFront(NodeSLL_2511533011 head_3011, int value_3011) {
        NodeSLL_2511533011 new_node = new NodeSLL_2511533011(value_3011);
        new_node.next_3011 = head_3011;
        return new_node;
    }

    public static NodeSLL_2511533011 insertAtEnd(NodeSLL_2511533011 head_3011, int value_3011) {
        NodeSLL_2511533011 newNode = new NodeSLL_2511533011(value_3011);
        if (head_3011 == null) {
            return newNode;
        }
        NodeSLL_2511533011 last_3011 = head_3011;
        while (last_3011.next_3011 != null) {
            last_3011 = last_3011.next_3011;
        }
        last_3011.next_3011 = newNode;
        return head_3011;
    }

    static NodeSLL_2511533011 GetNode(int data_3011) {
        return new NodeSLL_2511533011(data_3011);
    }

    static NodeSLL_2511533011 insertPos(NodeSLL_2511533011 headNode_3011,
                                        int position_3011, int value_3011) {
        NodeSLL_2511533011 head_3011 = headNode_3011;

        if (position_3011 < 1) {
            System.out.print("Invalid Position");
            return head_3011; // tambah return
        }
        if (position_3011 == 1) {
            NodeSLL_2511533011 new_node = new NodeSLL_2511533011(value_3011);
            new_node.next_3011 = head_3011; // fix hyperlink
            return new_node;
        } else {
            while (position_3011-- != 1) {
                if (position_3011 == 1) {
                    NodeSLL_2511533011 newNode = GetNode(value_3011);
                    newNode.next_3011 = headNode_3011.next_3011;
```

```java
                    headNode_3011.next_3011 = newNode;
                    break;
                }
                headNode_3011 = headNode_3011.next_3011;
            }
            if (position_3011 != 1)
                System.out.print("Posisi di luar jangkauan");
            return head_3011;
        }
    }

    public static void printList(NodeSLL_2511533011 head_3011) {
        NodeSLL_2511533011 curr_3011 = head_3011;
        while (curr_3011 != null) {
            System.out.print(curr_3011.data_3011);
            if (curr_3011.next_3011 != null) System.out.print(" --> ");
            curr_3011 = curr_3011.next_3011;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        // ☑ fix: buat semua node secara eksplisit
        NodeSLL_2511533011 head_3011 = new NodeSLL_2511533011(2);
        head_3011.next_3011 = new NodeSLL_2511533011(3);
        head_3011.next_3011.next_3011 = new NodeSLL_2511533011(5);
        head_3011.next_3011.next_3011.next_3011 = new NodeSLL_2511533011(6);

        System.out.print("Senarai berantai awal : ");
        printList(head_3011);

        System.out.print("Tambah 1 simpul di depan : ");
        head_3011 = InsertAtFront(head_3011, 1);
        printList(head_3011);

        System.out.print("Tambah 1 simpul di belakang : ");
        head_3011 = insertAtEnd(head_3011, 7);
        printList(head_3011);

        System.out.print("Tambah 1 simpul ke posisi 4 : ");
        head_3011 = insertPos(head_3011, 4, 4);
        printList(head_3011);
```

Gambar 2.2.3

Class **TambahSLL_2511533011** adalah kelas inti yang mengimplementasikan berbagai teknik manipulasi struktur data *Single Linked List*, khususnya dalam hal penambahan simpul (*insertion*) pada berbagai posisi.

1. Method InsertAtFront (Penambahan di Depan)

Metode ini bertujuan untuk menyisipkan data baru sebagai elemen pertama (*head*) dari list.

- **Logika:** Sebuah *node* baru dibuat, kemudian penunjuk next dari *node* baru tersebut diarahkan ke head yang lama.

- **Efisiensi:** Sangat efisien dengan kompleksitas waktu **$O(1)$** karena tidak memerlukan penelusuran (*traversal*) seluruh list.

2. Method insertAtEnd (Penambahan di Belakang)

Metode ini menyisipkan data baru di posisi paling akhir dari rantai *list*.

- **Logika:** Program harus melakukan penelusuran dari head menggunakan perulangan while hingga menemukan *node* yang next-nya bernilai null (simpul terakhir). Setelah ditemukan, next dari simpul terakhir tersebut dihubungkan ke *node* baru.
- **Efisiensi:** Memiliki kompleksitas **$O(n)$** karena harus melewati semua elemen yang ada.

3. Method insertPos (Penambahan di Posisi Tertentu)

Metode ini jauh lebih kompleks karena memungkinkan penyisipan data di tengah-tengah list berdasarkan nomor urut (posisi).

- **Logika:** Program menghitung langkah hingga mencapai satu posisi sebelum target. Penunjuk next dari *node* baru diarahkan ke *node* sesudahnya, dan *node* sebelumnya diarahkan ke *node* baru tersebut.
- **Penanganan Error:** Terdapat validasi jika posisi kurang dari 1 atau jika posisi yang diminta melebihi panjang list yang tersedia.

4. Method GetNode & printList

- **GetNode:** Berperan sebagai fungsi pembantu (*helper function*) untuk menciptakan objek *node* baru secara ringkas.
- **printList:** Berfungsi untuk menampilkan isi list ke layar dengan format visual menggunakan simbol panah (-->), memudahkan pengguna melihat keterhubungan antar simpul secara logis.

5. Method main (Driver Method)

Di sini dilakukan skenario pengujian lengkap:

1. Membuat list awal: 2 --> 3 --> 5 --> 6.
2. Menguji InsertAtFront: Menambahkan angka **1** di depan.
3. Menguji insertAtEnd: Menambahkan angka **7** di belakang.
4. Menguji insertPos: Menambahkan angka **4** tepat pada posisi ke-4.

2.2.4 **Class HapusSLL_2511533011**

```java
package Pekan5_2511533011;

public class HapusSLL_2511533011 {

    // Fungsi untuk menghapus head
    public static NodeSLL_2511533011 deleteHead_3011(NodeSLL_2511533011 head_3011) {
        // Jika SLL kosong
        if (head_3011 == null)
            return null;
        // Pindahkan head ke node berikutnya
        head_3011 = head_3011.next_3011;
        // Return head baru
        return head_3011;
    }

    // Fungsi menghapus node terakhir SLL
    public static NodeSLL_2511533011 removeLastNode_3011(NodeSLL_2511533011 head_3011) {
        // Jika list kosong, return null
        if (head_3011 == null) {
            return null;
        }
        // Jika list satu node, hapus node dan return null
        if (head_3011.next_3011 == null) {
            return null;
        }
        // Temukan node terakhir ke data
        NodeSLL_2511533011 secondLast_3011 = head_3011;
        while (secondLast_3011.next_3011.next_3011 != null) {
            secondLast_3011 = secondLast_3011.next_3011;
        }
        // Hapus node terakhir
        secondLast_3011.next_3011 = null;
        return head_3011;
    }

    // Fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511533011 deleteNode_3011(NodeSLL_2511533011 head_3011, int position_3011) {
        NodeSLL_2511533011 temp_3011 = head_3011;
        NodeSLL_2511533011 prev_3011 = null;

        // Jika linked list null
        if (temp_3011 == null) {
```

```java
            return head_3011;
        }

        // Kasus 1: hapus head (posisi 1)
        if (position_3011 == 1) {
            head_3011 = temp_3011.next_3011;
            return head_3011;
        }

        // Kasus 2: menghapus node di tengah
        // Telusuri ke node yang dihapus
        for (int i = 1; temp_3011 != null && i < position_3011; i++) {
            prev_3011 = temp_3011;
            temp_3011 = temp_3011.next_3011;
        }

        // Jika ditemukan, hapus node
        if (temp_3011 != null) {
            prev_3011.next_3011 = temp_3011.next_3011;
        } else {
            System.out.println("Data tidak ada");
        }
        return head_3011;
        }

    // Fungsi mencetak SLL
    public static void printList(NodeSLL_2511533011 head_3011) {
        NodeSLL_2511533011 curr_3011 = head_3011;
        while (curr_3011.next_3011 != null) {
            System.out.print(curr_3011.data_3011 + " --> ");
            curr_3011 = curr_3011.next_3011;
        }
            if (curr_3011.next_3011 == null) {
                System.out.print(curr_3011.data_3011); }
            System.out.println();
        }

    // Method main
    public static void main(String[] args) {
        // Buat SLL: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
        NodeSLL_2511533011 head_3011 = new NodeSLL_2511533011(1);
        head_3011.next_3011 = new NodeSLL_2511533011(2);
```

```java
        head_3011.next_3011.next_3011 = new NodeSLL_2511533011(3);
        head_3011.next_3011.next_3011.next_3011 = new NodeSLL_2511533011(4);
        head_3011.next_3011.next_3011.next_3011.next_3011 = new NodeSLL_2511533011(5);
        head_3011.next_3011.next_3011.next_3011.next_3011.next_3011 = new NodeSLL_2511533011(6);

        // Cetak list awal
        System.out.print("List awal : ");
        printList(head_3011);
        // hapus head
        head_3011 = deleteHead_3011(head_3011);
        System.out.print("List setelah head dihapus");
        printList(head_3011);
        // Hapus node terakhir
        head_3011 = removeLastNode_3011(head_3011);
        System.out.print("List setelah simpul terakhir dihapus : ");
        printList(head_3011);

        // Hapus node di posisi 2
        int position_3011 = 2;
        head_3011 = deleteNode_3011(head_3011, position_3011);
        System.out.print("List setelah posisi 2 dihapus : ");
        printList(head_3011);
    }
}
```

Gambar 2.2.4

Class **HapusSLL_2511533011** melengkapi operasi fundamental dalam struktur data dengan mengimplementasikan berbagai skenario penghapusan simpul (*deletion*) pada *Single Linked List*.

1. Method deleteHead_3011 (Penghapusan di Depan)

Metode ini digunakan untuk menghapus simpul pertama dari list.

- **Logika:** Penghapusan dilakukan dengan memindahkan penunjuk head ke simpul berikutnya (head.next). Simpul pertama yang lama secara otomatis akan terputus dari rantai.
- **Efisiensi:** Karena hanya mengubah satu referensi tanpa perlu menelusuri list, operasinya sangat cepat dengan kompleksitas **$O(1)$**. Di Java, *node* yang sudah tidak ditunjuk ini nantinya akan dibersihkan oleh *Garbage Collector*.

2. Method removeLastNode_3011 (Penghapusan di Belakang)

Metode ini bertujuan menghapus simpul paling akhir dari list.

- **Logika:** Program menggunakan variabel bantuan secondLast_3011 untuk menelusuri list hingga berhenti di **simpul penanda kedua terakhir**. Setelah ditemukan, pointer next dari simpul tersebut diatur menjadi null.
- **Kondisi Khusus:** Terdapat pengecekan jika list kosong atau hanya berisi satu simpul untuk menghindari kesalahan akses pointer (*NullPointerException*).

3. Method deleteNode_3011 (Penghapusan di Posisi Tertentu)

Metode ini memberikan fleksibilitas untuk menghapus simpul berdasarkan nomor urut posisi.

- **Logika:** Program melakukan iterasi untuk mencari simpul yang berada tepat sebelum posisi yang ingin dihapus (prev_3011). Penghapusan dilakukan dengan teknik "melompati" simpul target: prev.next = temp.next.

- **Skenario:**
    - **Kasus 1:** Jika posisi yang dihapus adalah 1, ia memanggil logika penghapusan *head*.
    - **Kasus 2:** Menghapus di tengah atau akhir list dengan menyambungkan simpul sebelumnya ke simpul setelah target.

4. Method printList & main

- **printList:** Menampilkan isi list saat ini agar perubahan setelah proses penghapusan dapat diamati secara langsung.
- **main:** Berperan sebagai skenario pengujian praktikum.
    1. Membentuk list awal: 1 --> 2 --> 3 --> 4 --> 5 --> 6.
    2. Menguji deleteHead_3011: Menghapus angka **1**.
    3. Menguji removeLastNode_3011: Menghapus angka **6**.
    4. Menguji deleteNode_3011: Menghapus angka pada posisi ke-2 (dalam sisa list).

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilaksanakan, dapat disimpulkan bahwa implementasi *Single Linked List* (SLL) menggunakan bahasa pemrograman Java berhasil memberikan solusi atas keterbatasan alokasi memori statis yang umumnya ditemukan pada struktur data *Array*. Melalui penggunaan class simpul yang dinamis, pengelolaan data menjadi lebih fleksibel karena memori hanya dialokasikan saat simpul baru diciptakan, sehingga mengoptimalkan penggunaan sumber daya sistem. Operasi fundamental seperti penambahan dan penghapusan simpul membuktikan bahwa manipulasi data pada SLL sangat bergantung pada pengaturan referensi objek, di mana operasi pada bagian depan (*head*) memiliki efisiensi tertinggi dibandingkan bagian tengah atau akhir yang memerlukan penelusuran sekuensial. Selain itu, integrasi dengan fitur *Garbage Collector* pada Java mempermudah manajemen memori dengan menghapus objek yang tidak lagi terjangkau secara otomatis, meskipun pencarian data tetap memerlukan waktu lebih lama karena ketiadaan fitur akses acak (*random access*).

3.2 Saran

Untuk pengembangan pemahaman di masa mendatang, penulis menyarankan agar praktikan mulai mengeksplorasi penggunaan pointer tambahan seperti *tail* untuk mengoptimalkan efisiensi penambahan data di akhir list menjadi waktu konstan. Selain itu, pengembangan ke arah struktur data yang lebih kompleks seperti *Double Linked List* atau *Circular Linked List* sangat dianjurkan guna mengatasi kendala penelusuran satu arah yang menjadi limitasi utama pada SLL.

DAFTAR PUSTAKA

[1] B. UNIVERSITY, "Single Linked List," School of Computer Science, [Online]. Available: https://socs.binus.ac.id/2017/03/15/single-linked-list/. [Accessed 06 Mei 2026].

LAPORAN TUGAS

Soal :

Pada tugas ini, mahasiswa diminta untuk mengimplementasikan konsep Single Linked List yang bekerja secara dinamis menggunakan pointer (referensi antar node). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian pasien, di mana setiap pasien direpresentasikan sebagai sebuah node yang memiliki pointer ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (FIFO — First In, First Out).

Jawaban :

1. Class Pasien_2511533011

```java
package Pekan5_2511533011;

public class Pasien_2511533011 {

    // Atribut
    private String namaPasien_3011;       // Nama pasien
    private String keluhan_3011;          // Keluhan / penyakit
    private int nomorAntrian_3011;        // Nomor antrian
    Pasien_2511533011 next_3011;          // Pointer ke node pasien berikutnya

    // Constructor
    public Pasien_2511533011(String namaPasien_3011, String keluhan_3011, int nomorAntrian_3011) {
        this.namaPasien_3011   = namaPasien_3011;
        this.keluhan_3011      = keluhan_3011;
        this.nomorAntrian_3011 = nomorAntrian_3011;
        this.next_3011         = null; // pointer awal selalu null
    }

    // Getter
    public String getNamaPasien_3011()  { return namaPasien_3011; }
    public String getKeluhan_3011()     { return keluhan_3011; }
    public int getNomorAntrian_3011()   { return nomorAntrian_3011; }
    public Pasien_2511533011 getNext_3011() { return next_3011; }

    // Setter
    public void setNamaPasien_3011(String namaPasien_3011)  { this.namaPasien_3011 = namaPasien_3011; }
    public void setKeluhan_3011(String keluhan_3011)        { this.keluhan_3011 = keluhan_3011; }
    public void setNomorAntrian_3011(int nomorAntrian_3011) { this.nomorAntrian_3011 = nomorAntrian_3011; }
    public void setNext_3011(Pasien_2511533011 next_3011)   { this.next_3011 = next_3011; }
}
```

Pasien_2511533011 adalah kelas **Node** yang merepresentasikan satu pasien dalam antrian. Kelas ini menyimpan data pasien seperti nama, keluhan, dan nomor antrian, serta memiliki pointer next_3011 yang menunjuk ke pasien berikutnya dalam linked list. Dilengkapi constructor, getter, dan setter untuk setiap atributnya.

2. Class RumahSakit_2511533011

```java
package Pekan5_2511533011;
import java.util.Scanner;

public class RumahSakit_2511533011 {

    // ATRIBUT SLL
    private Pasien_2511533011 head_3011;       // Pointer ke node terdepan (head)
    private Pasien_2511533011 tail_3011;       // Pointer ke node terakhir (tail)
    private int counterAntrian_3011;           // Counter auto-increment nomor antrian
    private int jumlahPasien_3011;             // Total pasien dalam antrian

    // CONSTRUCTOR
    public RumahSakit_2511533011() {
        head_3011           = null;
        tail_3011           = null;
        counterAntrian_3011 = 0;
        jumlahPasien_3011   = 0;
    }
    // 1. DAFTARKAN PASIEN — Insert at Tail
    public void daftarkanPasien_3011(String nama_3011, String keluhan_3011) {
        // Auto-increment nomor antrian
        counterAntrian_3011++;
        jumlahPasien_3011++;

        // Buat node baru
        Pasien_2511533011 newPasien_3011 = new Pasien_2511533011(nama_3011, keluhan_3011, counterAntrian_3011);

        // Jika list kosong, node baru langsung jadi head dan tail
        if (head_3011 == null) {
            head_3011 = newPasien_3011;
            tail_3011 = newPasien_3011;
        } else {
            // Sambungkan node terakhir ke node baru, lalu geser tail
            tail_3011.setNext_3011(newPasien_3011);
            tail_3011 = newPasien_3011;
        }

        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + counterAntrian_3011);
    }
    // 2. PANGGIL PASIEN — Delete Head
    public void panggilPasien_3011() {
        // Tangani kondisi list kosong
```

```java
        if (head_3011 == null) {
            System.out.println("Antrian kosong! Tidak ada pasien yang perlu dipanggil.");
            return;
        }

        // Simpan data head sebelum dihapus
        Pasien_2511533011 dipanggil_3011 = head_3011;

        // Geser head ke node berikutnya
        head_3011 = head_3011.getNext_3011();

        // Jika setelah dihapus list jadi kosong, reset tail juga
        if (head_3011 == null) {
            tail_3011 = null;
        }

        jumlahPasien_3011--;

        // Tampilkan data pasien yang dipanggil
        System.out.println("=== Pasien Dipanggil ===");
        System.out.println("Nomor Antrian : " + dipanggil_3011.getNomorAntrian_3011());
        System.out.println("Nama          : " + dipanggil_3011.getNamaPasien_3011());
        System.out.println("Keluhan       : " + dipanggil_3011.getKeluhan_3011());
    }
    // 3. TAMPILKAN ANTRIAN — Display (Traversal)
    public void tampilkanAntrian_3011() {
        // Tangani kondisi list kosong
        if (head_3011 == null) {
            System.out.println("Antrian kosong! Belum ada pasien terdaftar.");
            return;
        }

        System.out.println("==== Daftar Antrian Pasien ====");
        Pasien_2511533011 curr_3011 = head_3011; // Mulai dari head
        int posisi_3011 = 1;

        // Telusuri sampai pointer null
        while (curr_3011 != null) {
            System.out.println("Posisi " + posisi_3011 + " :");
            System.out.println("  No. Antrian : " + curr_3011.getNomorAntrian_3011());
            System.out.println("  Nama        : " + curr_3011.getNamaPasien_3011());
            System.out.println("  Keluhan     : " + curr_3011.getKeluhan_3011());
```

```java
            System.out.println("Antrian kosong! Tidak ada pasien saat ini.");
        } else {
            System.out.println("Total Pasien dalam Antrian : " + jumlahPasien_3011);
            System.out.println("--- Pasien Terdepan ---");
            System.out.println("  No. Antrian : " + head_3011.getNomorAntrian_3011());
            System.out.println("  Nama        : " + head_3011.getNamaPasien_3011());
            System.out.println("  Keluhan     : " + head_3011.getKeluhan_3011());
        }
    }
    // MAIN — Menu Interaktif
    public static void main(String[] args) {
        Scanner scanner_3011 = new Scanner(System.in);
        RumahSakit_2511533011 rs_3011 = new RumahSakit_2511533011();
        int pilihan_3011;

        do {
            // Tampilkan menu
            System.out.println("\n=== Antrian Rumah Sakit NIM: 2511533011 ===");
            System.out.println("1. Daftarkan Pasien (Insert)");
            System.out.println("2. Panggil Pasien (Delete Head)");
            System.out.println("3. Tampilkan Antrian (Display)");
            System.out.println("4. Cari Pasien (Search)");
            System.out.println("5. Cek Status Antrian");
            System.out.println("6. Keluar");
            System.out.print("Pilihan: ");
            pilihan_3011 = scanner_3011.nextInt();
            scanner_3011.nextLine(); // consume newline

            switch (pilihan_3011) {
                case 1:
                    System.out.print("Masukkan Nama Pasien : ");
                    String nama_3011 = scanner_3011.nextLine();
                    System.out.print("Masukkan Keluhan     : ");
                    String keluhan_3011 = scanner_3011.nextLine();
                    rs_3011.daftarkanPasien_3011(nama_3011, keluhan_3011);
                    break;
                case 2:
                    rs_3011.panggilPasien_3011();
                    break;
                case 3:
                    rs_3011.tampilkanAntrian_3011();
                    break;
```

```java
            System.out.println("  Keluhan     : " + curr_3011.getKeluhan_3011());
            if (curr_3011.getNext_3011() != null) System.out.println("       |");
            curr_3011 = curr_3011.getNext_3011(); // Geser pointer ke node berikutnya
            posisi_3011++;
        }
    }
    // 4. CARI PASIEN — Search (Case-Insensitive)
    public void cariPasien_3011(String namaCari_3011) {
        // Tangani kondisi list kosong
        if (head_3011 == null) {
            System.out.println("Antrian kosong! Tidak ada pasien untuk dicari.");
            return;
        }

        Pasien_2511533011 curr_3011 = head_3011;
        boolean ditemukan_3011 = false;

        System.out.println("=== Hasil Pencarian: \"" + namaCari_3011 + "\" ===");

        // Telusuri seluruh list
        while (curr_3011 != null) {
            // Perbandingan case-insensitive menggunakan toLowerCase()
            if (curr_3011.getNamaPasien_3011().toLowerCase().contains(namaCari_3011.toLowerCase())) {
                System.out.println("  Pasien ditemukan!");
                System.out.println("  No. Antrian : " + curr_3011.getNomorAntrian_3011());
                System.out.println("  Nama        : " + curr_3011.getNamaPasien_3011());
                System.out.println("  Keluhan     : " + curr_3011.getKeluhan_3011());
                ditemukan_3011 = true;
            }
            curr_3011 = curr_3011.getNext_3011();
        }

        // Tampilkan pesan jika tidak ditemukan
        if (!ditemukan_3011) {
            System.out.println("  Pasien dengan nama \"" + namaCari_3011 + "\" tidak ditemukan.");
        }
    }
    // 5. CEK STATUS ANTRIAN
    public void cekStatusAntrian_3011() {
        System.out.println("=== Status Antrian ===");

        if (head_3011 == null) {
```

```java
                case 4:
                    System.out.print("Masukkan Nama yang dicari : ");
                    String cari_3011 = scanner_3011.nextLine();
                    rs_3011.cariPasien_3011(cari_3011);
                    break;
                case 5:
                    rs_3011.cekStatusAntrian_3011();
                    break;
                case 6:
                    System.out.println("Terima kasih! Program selesai.");
                    break;
                default:
                    System.out.println("Pilihan tidak valid! Masukkan angka 1-6.");
            }

        } while (pilihan_3011 != 6);

        scanner_3011.close();
    }
}
```

RumahSakit_2511533011 adalah kelas **Driver** yang mengelola keseluruhan sistem antrian menggunakan Single Linked List dengan pola FIFO. Kelas ini memiliki pointer head (pasien terdepan) dan tail (pasien terakhir), serta lima operasi utama:

- **Daftarkan Pasien** — menambah pasien baru di belakang antrian
- **Panggil Pasien** — menghapus dan menampilkan pasien terdepan

- **Tampilkan Antrian** — menelusuri semua pasien dari head ke tail
- **Cari Pasien** — mencari pasien berdasarkan nama (tidak sensitif huruf besar/kecil)
- **Cek Status** — menampilkan jumlah pasien dan info pasien terdepan

Output :

```
=== Antrian Rumah Sakit NIM: 2511533011 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : Budi Santoso
Masukkan Keluhan     : Demam
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511533011 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan:
```